Measuring the carbon footprint of Python algorithms

Green Software — Universidad Tecnológica Nacional - FRC

Run date: 2026-09-01 19:35:49

> Every figure in this report comes from the measurements recorded in the run shown above. None was written by hand or taken from another source.

---

1. Objective and hypothesis

Objective. To measure the carbon footprint associated with running algorithms in Python and to determine whether algorithmic design decisions have a quantifiable environmental consequence.

Hypothesis. Two implementations that produce exactly the same result can differ by orders of magnitude in the energy they consume and, therefore, in the CO_2 they emit. Computational complexity is not just a theoretical property: it translates into electricity consumption and emissions.

Experimental design. Four experiments, each with variants that solve the same problem and return the same result (equivalence verified by assertion before each measurement):

- **Sorting algorithms** — `bubble_sort` ($O(n^2)$), `merge_sort` ($O(n \log n)$), `timsort_nativo` ($O(n \log n)$ in C)
- **Computing Fibonacci** — `fib_recursivo` ($O(\varphi^n)$), `fib_memoizado` ($O(n)$), `fib_iterativo` ($O(n)$)
- **Matrix multiplication** — `matmul_python` ($O(n^3)$ in Python), `matmul_numpy` ($O(n^3)$ vectorized)
- **Training and inference of an ML model (KNN)** — `knn_brute` ($O(n \cdot m \cdot d)$ vectorized), `knn_kd_tree` ($O(n \cdot d) + O(n \log n)$), `knn_ball_tree` ($O(n \cdot d) + O(n \log n)$)

2. Theoretical framework

The carbon footprint of a computation is obtained by multiplying the electrical energy consumed by the carbon intensity of the grid that supplies it:

```
CO₂ [kgCO₂eq]  =  E [kWh]  ×  EF [kgCO₂eq/kWh]  ×  PUE
```

where EF is the emission factor of the power grid and PUE (*Power Usage Effectiveness*) accounts for the supporting infrastructure, such as the cooling of a data center.

Emission factor of the Argentine grid

The adopted value is **EF = 346.0 gCO2eq/kWh = 0.346 kgCO2eq/kWh**, corresponding to 2025, the most recent year available for Argentina.

Source: Ember (2026), Yearly Electricity Data — with processing by Our World in Data (https://ourworldindata.org/grapher/carbon-intensity-electricity, downloaded on 2026-08-22). The indicator is the *life-cycle carbon intensity of electricity generation*. The full series is versioned in `datos/intensidad_carbono_argentina.json` and the project selects the latest available year on every run, so the value can neither go out of date nor be written by hand.

Recent evolution of the carbon intensity of the Argentine grid:

Year	gCO2eq/kWh
2020	394.70
2021	406.99
2022	392.69
2023	350.90
2024	344.83
2025	346.00

Why the factor bundled with the measurement tool is not used. CodeCarbon ships its own table of factors per country, but for Argentina it is frozen at 2023: 354.103 gCO2eq/kWh, even in the latest published version of the tool (3.3.0). That value is 2.3% higher than the 2025 figure and would be out of date.

That is why this report takes the energy measured by the tool and converts it to CO_2 with the 2025 factor, the most recent available. The figure the tool would compute with its own table is also recorded in the run data, so that the difference can be verified.

As an additional cross-check, Secretaría de Energía de la Nación (Argentina's national Energy Secretariat) publishes a grid emission factor using the UNFCCC, Tool to calculate the emission factor for an electricity system, v7 methodology: its latest available value is **343.5 gCO2eq/kWh** for 2023 (http://datos.energia.gob.ar/dataset/calculo-del-factor-de-emision-de-co2-de-la-red-argentina-de-energia-electrica, file `factor_de_emision_2013_a_2023.xlsx`, downloaded on 2026-08-22). It is not adopted here because it is a marginal factor, intended for project evaluation, and not an average grid intensity, which is what applies when attributing the consumption of an arbitrary process. Its closeness to the adopted value (0.7% difference) nonetheless serves as an order-of-magnitude check.

The order of magnitude of the factor is explained by the composition of the Argentine electricity mix in 2023, the latest year with a breakdown by source in the data shipped with CodeCarbon: out of a total of 153.4 TWh generated, **59.5% comes from fossil sources** (gas 73.4 TWh, oil 17.3 TWh, coal 0.6 TWh), compared with 33.2 TWh of hydroelectric, 8.9 TWh of nuclear, 14.5 TWh of wind and 3.2 TWh of solar power.

A relevant consequence for the analysis: **the same algorithm has a different footprint depending on where it runs**. Running it on a mostly hydroelectric or nuclear grid, emissions would be a fraction of those calculated here.

PUE = 1.0 is used because the experiment runs on a laptop, with no external cooling infrastructure. In a data center the typical PUE is between 1.1 and 1.6 (an industry reference range, not measured in this work), which would multiply all results by that factor.

3. Methodology

3.1 Measurement environment

Parameter	Value
Platform	macOS-26.6.2-arm64-arm-64bit
Processor	Apple M2 Pro
Cores	12 logical / 12 physical
RAM	16.0 GB
Python	3.12.9
NumPy	2.5.2
CodeCarbon	3.3.0
scikit-learn	1.9.0
Cores available to the numerical library	12
Environment	exact versions pinned, so that the run is repeatable
Random seed	42
Repetitions per point	3

3.2 Measurement protocol

Each measurement point is a combination of algorithm and input size, and corresponds to one row of the tables in section 4. With four experiments and their variants there are 42 points. Each one is measured as follows:

1. **Equivalence is verified.** Before measuring, an assertion checks that all variants of the experiment return the same result. Comparing the footprint of algorithms that do not solve the same problem would not be valid.
2. **The batch size is computed.** Several of these algorithms finish in microseconds, far less than a sensor needs to produce a power reading, so a single isolated run is not measured: what is measured is a **batch of K consecutive runs**, with K chosen so that the batch lasts at least 3.0 s (section 3.4). K is computed with a preliminary trial run, which also serves as a warm-up: the first run of an algorithm is always slower than the following ones.
3. **The batch is measured and divided by K.** What is reported is therefore the cost of *one* run, even though no single run was ever measured on its own. When an algorithm already lasts longer than the window by itself, as model training does at the larger sizes, K stays at 1.

4. **It is repeated 3 times** and the mean and spread across those 3 measurements are reported.

A concrete example from this run, to make the idea tangible: the fastest algorithm in the benchmark, `fib_iterativo`, was measured in batches of 5,623,564 runs; with 3 repetitions, behind that row of the table there are 16,870,692 actual runs of the algorithm.

3.3 Another way to calculate: a custom model

There is an alternative to using a measurement tool: deriving power from how much processor the algorithm uses, and attributing to it only the additional consumption it causes. The calculation chain would be:

```
processor usage     =  CPU time consumed / (elapsed time × cores)
attributed power    =  (maximum power − idle power) × usage  +  memory × W/GB
energy              =  power × time
footprint           =  energy × emission factor × PUE
```

It requires three parameters of the machine it runs on: idle power, power with all cores busy, and memory consumption per GB used. The first two must be measured with a power sensor; assuming them makes the result as debatable as the assumption itself.

It is mentioned as an option and nothing more: this work does not implement it. Everything that follows comes from the approach that is actually developed.

3.4 The approach of this work: CodeCarbon

CodeCarbon is the industry reference tool for measuring the carbon footprint of software. Every figure in sections 4 and 5 comes from it. It was used according to three criteria:

- **The country is declared, not inferred.** The tool can work it out from the internet connection, but that would tie the result to the network it is run from and to a particular moment. Here Argentina is set up front, and the run stays repeatable.
- **A single measurement session for the whole benchmark.** Starting the tool takes time, because it first identifies the hardware: 2.8 s in this run. If it were started and stopped for every measurement, that time would fall inside what is measured and would ruin any measurement in the millisecond range. It is opened only once and each algorithm is measured as a task within that session.
- **Power is measured, and only estimated when the machine does not allow measuring it.** This is the decision with the greatest weight on the results, which is why it has its own section (section 5.2). Some machines expose a sensor of the processor's actual power draw; where it exists and is accessible, power is read from it. Where not, the tool calculates it with a formula. This software can run on another machine: the path is decided on each run according to what that machine allows, and the report states which one was used instead of taking it for granted.

In this run, power was measured with the machine's sensor. The CO_2 values that follow therefore rest on actual consumption and not on an estimate.

Measuring actual consumption places an additional demand on the protocol: the reading has to be taken while the algorithm is working. That is why the batches in section 3.2 are sized to last at least 3.0 s, instead of the 0.5 s that would suffice for timing. With shorter windows the reading arrives after the work has finished, and the reported consumption is inflated.

4. Results

4.0 How to read the tables

Each row is a measurement point (section 3.2) and the values are the mean of its 3 measurements, expressed per run of the algorithm.

Column	What it is
`n`	input size, in the units declared by each experiment
`algorithm`	the measured variant; all variants of the experiment return the same result
`complexity`	the theoretical complexity of the algorithm
`time [s]`	how long one run takes
`energy [kWh]`	how much electricity one run consumes. This is the quantity being measured (section 3.4)
`CO₂ [kg]`	that energy converted to emissions with the 0.346 kgCO2eq/kWh factor from section 2
`spread`	how much the repetitions of the point differ from each other: the standard deviation as a percentage of the mean

On spread: below 1% the measurement is reproducible. A high value does not invalidate the mean, and it can come from two places: from the variants that split their work across several cores, where how many get activated changes from one repetition to the next, or from the time it takes to read the power consumption (section 5.2), which weighs more on the fastest algorithms.

What a row says, exactly. Taking the first row of the first experiment (`bubble_sort`, number of elements = 1,000), each number means the following:

- `n` = **1,000**: the input it was run on had that size.
- `complexity` = **O(n²)**: theoretical label declared by the project. It was neither measured nor derived from the data.
- `time [s]` = **2.866e-02**: the algorithm was run 105 times in a row and the whole set was timed; the total was divided by 105. This was done 3 times and the 3 quotients were averaged.
- `energy [kWh]` = **2.671e-08**: during those same 105 runs, the power read from the processor plus the power attributed to memory was integrated over the whole window. The total was divided by 105 and the 3 measurements were averaged.
- `CO₂ [kg]` = **9.243e-09**: the energy above multiplied by 0.346. It is not an independent measurement: it is the same figure in another unit.
- `spread` = **±15.2%**: the standard deviation of the 3 measured CO2 values (1.087e-08, 8.450e-09, 8.411e-09), divided by their mean.

Sorting algorithms

Question: Does computational complexity translate into a measurable difference in emissions when the output produced is identical?

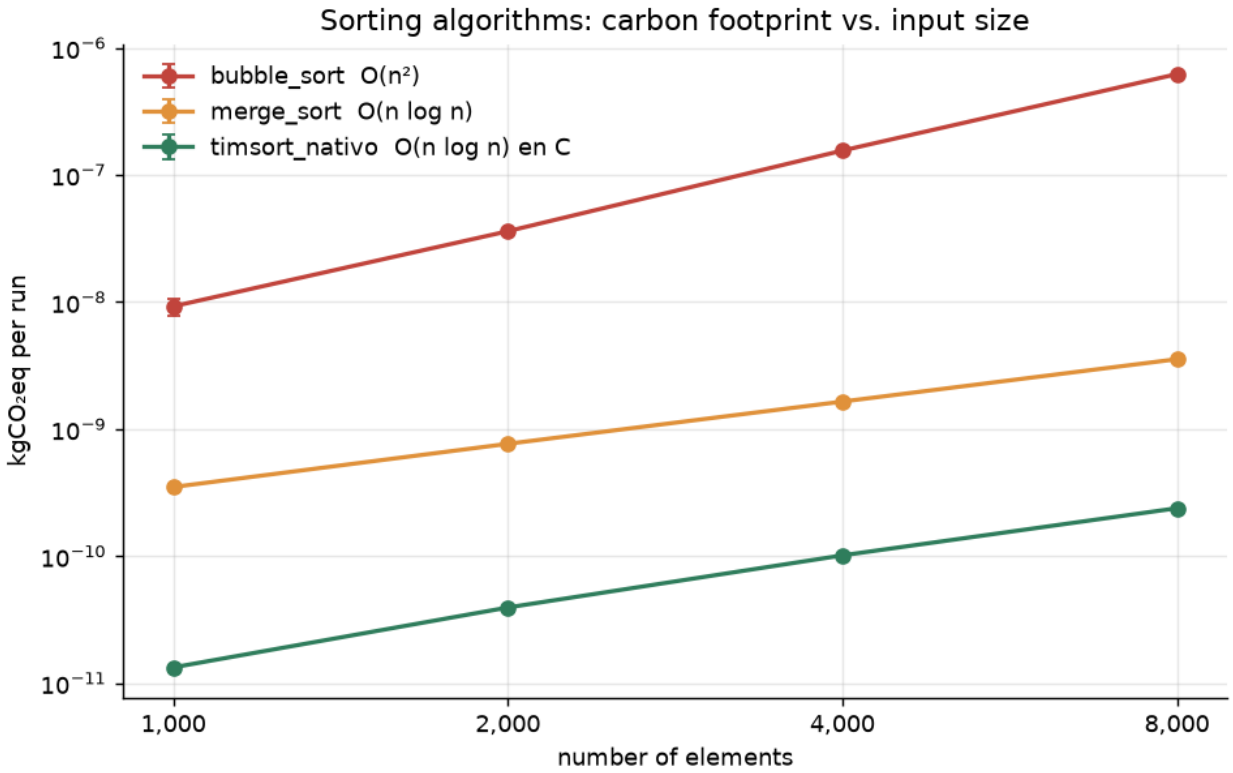
All three variants sort exactly the same list of integers, generated with a fixed seed, and return the same result.

n	algorithm	complexity	time [s]	energy [kWh]	CO_2 [kg]	spread
1000	`bubble_sort`	O(n²)	2.866e-02	2.671e-08	9.243e-09	±15.2%
1000	`merge_sort`	O(n log n)	1.178e-03	1.011e-09	3.498e-10	±1.0%
1000	`timsort_nativo`	O(n log n) in C	4.507e-05	3.848e-11	1.331e-11	±1.1%
2000	`bubble_sort`	O(n²)	1.218e-01	1.041e-07	3.603e-08	±0.5%
2000	`merge_sort`	O(n log n)	2.557e-03	2.211e-09	7.650e-10	±1.6%
2000	`timsort_nativo`	O(n log n) in C	1.329e-04	1.139e-10	3.942e-11	±3.9%
4000	`bubble_sort`	O(n²)	5.222e-01	4.488e-07	1.553e-07	±2.7%
4000	`merge_sort`	O(n log n)	5.512e-03	4.752e-09	1.644e-09	±2.0%
4000	`timsort_nativo`	O(n log n) in C	3.432e-04	2.928e-10	1.013e-10	±1.4%
8000	`bubble_sort`	O(n²)	2.107e+00	1.795e-06	6.209e-07	±0.3%
8000	`merge_sort`	O(n log n)	1.186e-02	1.025e-08	3.545e-09	±2.2%
8000	`timsort_nativo`	O(n log n) in C	7.995e-04	6.883e-10	2.382e-10	±2.4%

Result for n = 8,000: `timsort_nativo` (O(n log n) in C) emits **2,607 times less** CO_2 than `bubble_sort` (O(n²)), while producing exactly the same result.

What each algorithm means.

- **`bubble_sort`** — Compares neighboring elements and swaps them if they are out of order, repeating the pass until there is nothing left to swap. It is the simplest method to write and the one that does the most work.
- **`merge_sort`** — Splits the list in halves until only single elements remain, then merges them back in pairs, already sorted. Written in Python.
- **`timsort_nativo`** — Python's built-in sort, written in C. It uses the same idea of merging sorted halves, but also detects and takes advantage of stretches that were already in order.



Computing Fibonacci

Question: What is the environmental cost of recomputing work already done? A comparison between exponential and linear growth.

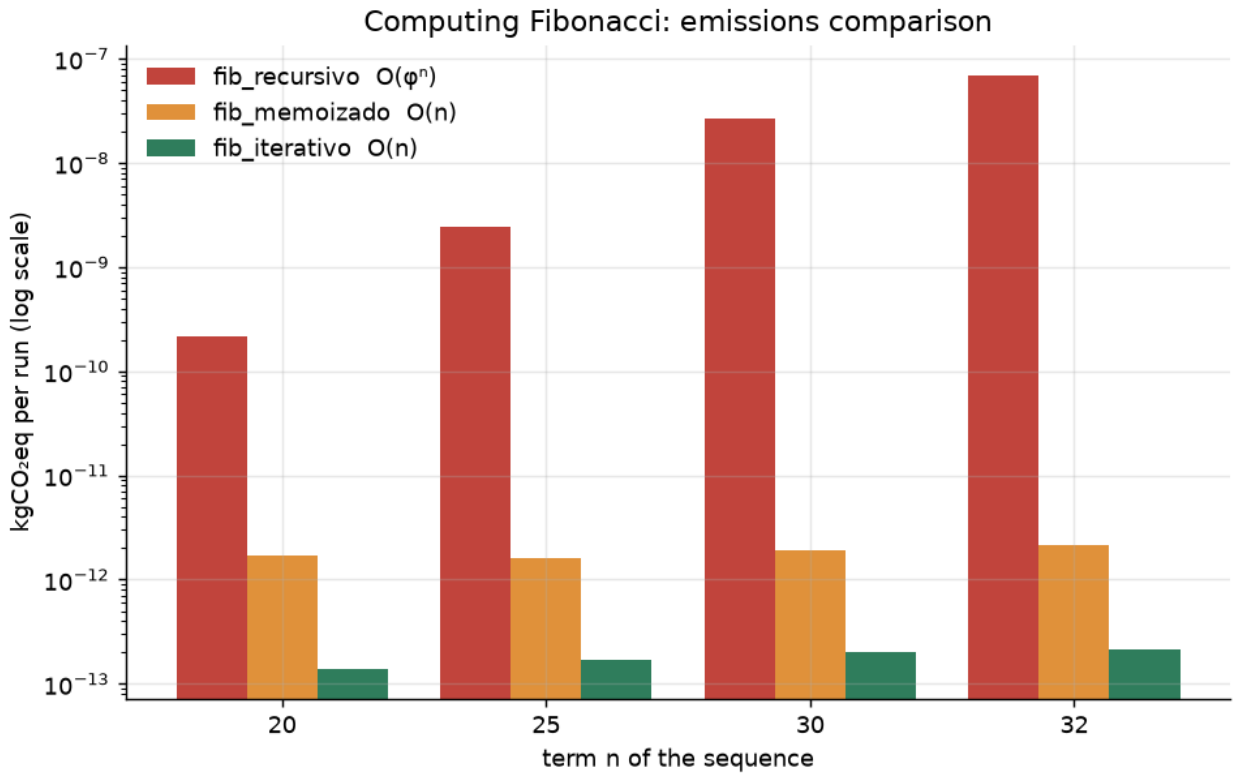
Capped at n=32 because the naive recursive variant grows exponentially and would dominate the total benchmark time.

n	algorithm	complexity	time [s]	energy [kWh]	CO_2 [kg]	spread
20	`fib_iterativo`	O(n)	4.493e-07	4.010e-13	1.387e-13	±2.1%
20	`fib_memoizado`	O(n)	5.271e-06	4.934e-12	1.707e-12	±10.8%
20	`fib_recursivo`	$O(\varphi^n)$	7.353e-04	6.286e-10	2.175e-10	±0.1%
25	`fib_iterativo`	O(n)	5.642e-07	4.906e-13	1.697e-13	±1.9%
25	`fib_memoizado`	O(n)	5.474e-06	4.667e-12	1.615e-12	±0.3%
25	`fib_recursivo`	$O(\varphi^n)$	8.154e-03	7.057e-09	2.442e-09	±3.0%
30	`fib_iterativo`	O(n)	6.769e-07	5.827e-13	2.016e-13	±1.3%
30	`fib_memoizado`	O(n)	6.466e-06	5.505e-12	1.905e-12	±0.6%
30	`fib_recursivo`	$O(\varphi^n)$	9.036e-02	7.711e-08	2.668e-08	±0.5%
32	`fib_iterativo`	O(n)	7.186e-07	6.150e-13	2.128e-13	±1.7%
32	`fib_memoizado`	O(n)	6.774e-06	6.213e-12	2.150e-12	±8.2%
32	`fib_recursivo`	$O(\varphi^n)$	2.361e-01	2.020e-07	6.989e-08	±0.6%

Result for n = 32: `fib_iterativo` (O(n)) emits **328,443 times less** CO_2 than `fib_recursivo` ($O(\varphi^n)$), while producing exactly the same result.

What each algorithm means.

- **`fib_recursivo`** — To get a term it adds the two previous ones, and for each of them it starts over from scratch, remembering nothing. It ends up recomputing the same values an enormous number of times.
- **`fib_memoizado`** — Does the same, but records each result the first time it computes it and then reuses it instead of computing it again.
- **`fib_iterativo`** — Walks through the sequence just once, from the bottom up, keeping only the last two values.



Matrix multiplication

Question: With the same $O(n^3)$ algorithmic complexity, how much does the choice of implementation (pure Python versus an optimized numerical library) weigh environmentally?

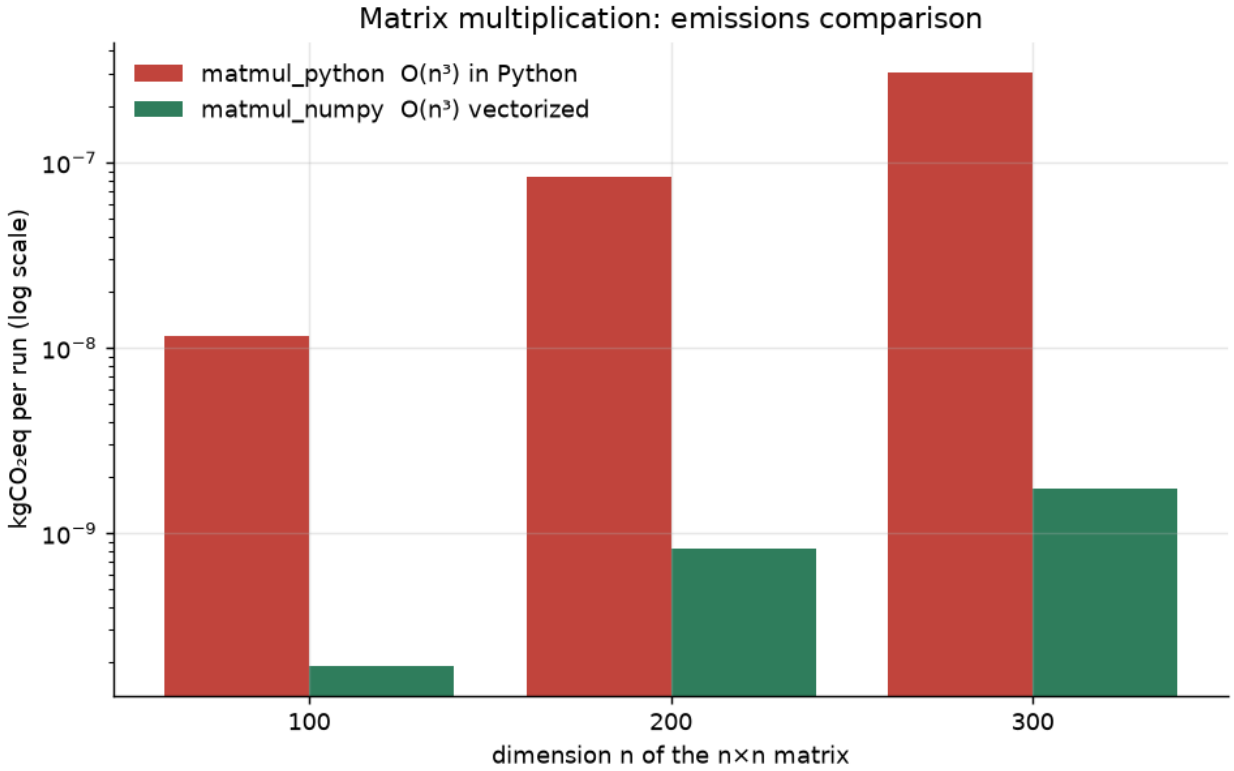
Both variants compute the same product with the same asymptotic complexity. The difference in footprint is purely the cost of implementation.

n	algorithm	complexity	time [s]	energy [kWh]	CO_2 [kg]	spread
100	`matmul_numpy`	O(n³) vectorized	6.488e-04	5.542e-10	1.917e-10	±0.6%
100	`matmul_python`	O(n³) in Python	3.573e-02	3.364e-08	1.164e-08	±11.6%
200	`matmul_numpy`	O(n³) vectorized	2.539e-03	2.399e-09	8.299e-10	±17.7%
200	`matmul_python`	O(n³) in Python	2.800e-01	2.405e-07	8.321e-08	±0.7%
300	`matmul_numpy`	O(n³) vectorized	5.827e-03	4.989e-09	1.726e-09	±1.1%
300	`matmul_python`	O(n³) in Python	9.818e-01	8.904e-07	3.081e-07	±7.3%

Result for n = 300: `matmul_numpy` (O(n³) vectorized) emits **178 times less** CO_2 than `matmul_python` (O(n³) in Python), while producing exactly the same result.

What each algorithm means.

- **`matmul_python`** — Multiplies the matrices element by element, with three nested loops written in Python. Every operation goes through the interpreter.
- **`matmul_numpy`** — Hands the same multiplication to a compiled numerical library, which operates on blocks of data at once instead of number by number.



Training and inference of an ML model (KNN)

Question: With exactly the same predictive result, how much does the footprint change depending on how the neighbor search is implemented, and what happens when one of them uses all cores instead of one?

The model is a nearest-neighbors classifier, which does not infer any rule: to decide which category a new data point belongs to, it finds the 31 most similar points among those it already knows and

assigns it the category that prevails among those 31. Here, similar means close, measuring the distance across the 20 features that describe each data point.

The expensive part is precisely finding those 31 neighbors among thousands of points, and that is where the three variants differ. Training and classification are measured as a single unit of work, and all three return exactly the same predictions.

n	algorithm	complexity	time [s]	energy [kWh]	CO_2 [kg]	spread
2500	`knn_ball_tree`	O(n·d) + O(n log n)	1.934e-02	2.020e-08	6.989e-09	±17.6%
2500	`knn_brute`	O(n·m·d) vectorized	4.086e-03	4.632e-09	1.603e-09	±36.2%
2500	`knn_kd_tree`	O(n·d) + O(n log n)	2.262e-02	3.234e-08	1.119e-08	±8.9%
5000	`knn_ball_tree`	O(n·d) + O(n log n)	6.193e-02	5.690e-08	1.969e-08	±2.9%
5000	`knn_brute`	O(n·m·d) vectorized	9.305e-03	9.911e-09	3.429e-09	±10.9%
5000	`knn_kd_tree`	O(n·d) + O(n log n)	5.901e-02	5.781e-08	2.000e-08	±5.5%
10000	`knn_ball_tree`	O(n·d) + O(n log n)	2.234e-01	1.951e-07	6.750e-08	±1.6%
10000	`knn_brute`	O(n·m·d) vectorized	2.082e-02	1.956e-08	6.769e-09	±9.6%
10000	`knn_kd_tree`	O(n·d) + O(n log n)	2.399e-01	2.136e-07	7.390e-08	±1.2%
20000	`knn_ball_tree`	O(n·d) + O(n log n)	8.246e-01	7.688e-07	2.660e-07	±4.3%
20000	`knn_brute`	O(n·m·d) vectorized	5.425e-02	5.711e-08	1.976e-08	±7.6%
20000	`knn_kd_tree`	O(n·d) + O(n log n)	6.872e-01	6.652e-07	2.302e-07	±8.9%

Result for n = 20,000: `knn_brute` (O(n·m·d) vectorized) emits **13.5 times less** CO_2 than `knn_ball_tree` (O(n·d) + O(n log n)), while producing exactly the same result.

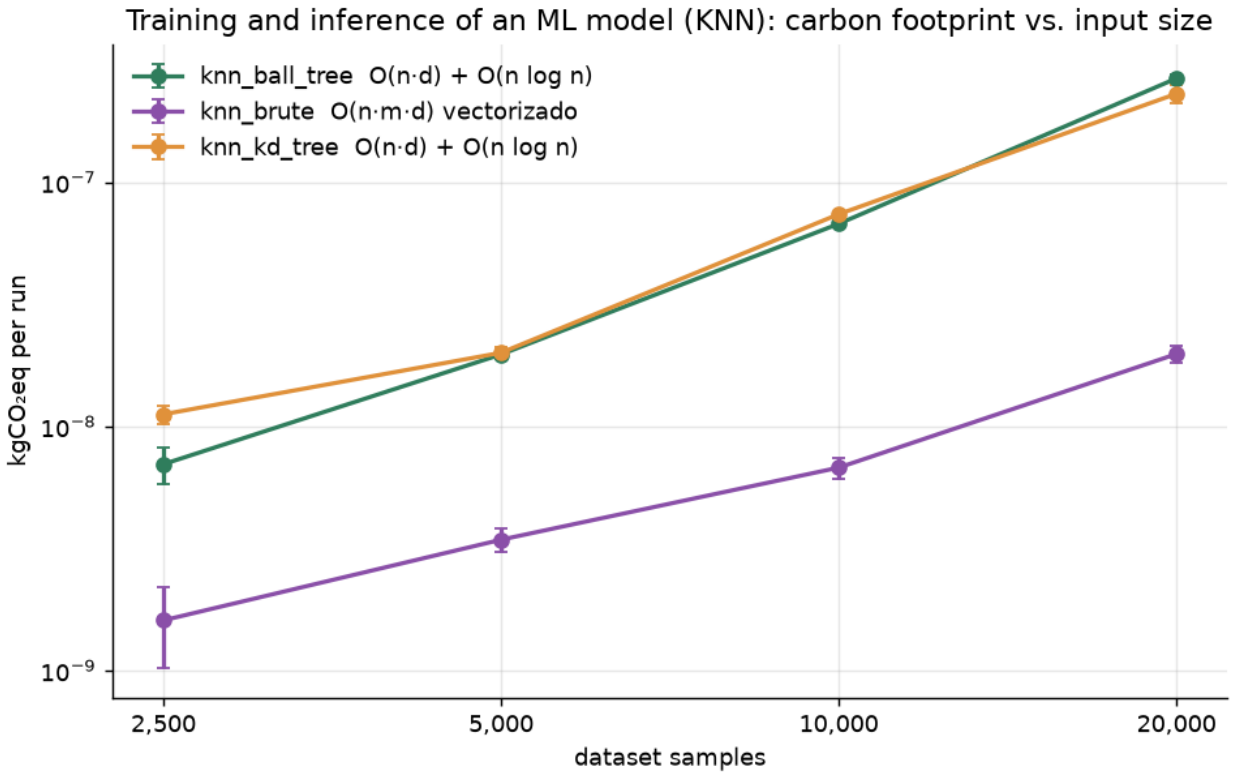
Model quality. The comparison is between equivalent implementations, not between different models: the three variants return the same predictions and therefore the same accuracy. It is reported to show that the measured work is a model that actually learns, and not an empty computation.

samples	accuracy on the test set
2,500	96.80%
5,000	95.30%
10,000	92.20%
20,000	96.28%

What each algorithm means.

- **`knn_brute`** — Measures the distance from the new point to every known point, one by one, and keeps the 31 closest. It discards nothing, but it does those computations in blocks and spreads the work across all cores.
- **`knn_kd_tree`** — Before classifying, it arranges the known points by splitting the space with straight cuts, like an irregular grid. When searching for neighbors it discards the far-away zones in one go, instead of going point by point.

- **`knn_ball_tree`** — Does the same as `knn_kd_tree` (discarding whole zones instead of going through everything), but groups the known points into bubbles instead of cutting them with straight lines. Uses a single core.



5. Analysis

5.1 Execution time as a proxy for the footprint

Across the 42 configurations measured, emissions turn out to be almost exactly proportional to execution time: relating both quantities, an algorithm that takes twice as long emits 2.01 times more, and the relationship explains 99.95% of the observed variation.

It is worth understanding why, because it is not a general law. Almost all workloads in this benchmark use the processor the same way: a single core, computing without pauses. At the same usage intensity, consumption per second is similar, and so energy, which is consumption over time, ends up governed by time. **The proportionality is real, but it holds among workloads that use the machine in a comparable way.**

Practical conclusion: in this benchmark execution time turned out to be a good indicator of the footprint, even for the workload that uses every core. It should not be generalized: a program that spends its time waiting on disk or network accumulates seconds while consuming almost nothing, and there the relationship breaks down. This work does not measure that case.

How to read the chart below.

Each point is **one algorithm solving a problem of a given size**. Sorting a list of 8,000 elements with `bubble_sort` is one point; sorting one of 1,000 with the same algorithm is a different point. There are 42 in total, one for each combination of algorithm and size that was measured.

The position of each point says two things: **the further right it is, the longer** that run takes, and **the higher it is, the more CO_2 it emits**. So the bottom-left corner is where the fast, clean algorithms are, and the top-right corner is where the slow, dirty ones are.

Both axes advance **by multiplying by ten at each tick**, instead of adding a fixed amount. This is needed because the durations are very different from each other: `fib_iterativo` takes 449.30 nanoseconds and `bubble_sort` takes 2.11 seconds, that is, 4,690,157 times longer. On an ordinary scale, all the fast points would be piled up against the left edge and nothing could be seen.

And what to look for is that **the points form a straight line**, without scattering: the algorithm that takes twice as long emits about twice as much, and this holds across the whole measured range, from microseconds up to two seconds. If time and emissions were unrelated, the points would appear scattered across the whole chart instead of lined up.



5.2 How much confidence these figures deserve

A software carbon footprint is energy converted to CO_2, and energy is power times time. Time is timed without difficulty. Power is the hard part, and everything else depends on it, so its origin must be stated.

In this run power was measured with the machine's own power sensor (section 3.4). The absolute CO_2 values therefore rest on actual consumption. Two caveats remain, which concern not the calculation but the scope of the measurement:

1. **The sensor measures the whole processor, not the algorithm.** If another program is working at the same time, part of its consumption is attributed to the measured algorithm. Measurements were taken with the machine idle and each point repeated three times, but it is not entirely eliminated.
2. **Taking the reading takes time of its own.** The reported consumption covers the entire measurement window, and that window includes the time the instrument takes to read: 1.9 s per measurement in this run, over windows of 3.0 s. That time passes with the algorithm already finished and the machine nearly idle, so **it adds some consumption that does not belong to the algorithm**. It affects the fastest algorithms the most, since they are the ones with the shortest windows.

The alternative would have been to estimate it. When a machine does not expose an accessible sensor, the tool infers power from the processor's nominal consumption and from how busy the machine is. It is worth knowing what was given up by not taking that path: on this machine the tool does not recognize the processor, so it would have assigned it a generic nominal consumption of 85 W (that of a desktop computer, not a laptop) and derived all the energy from there.

What can and cannot be claimed.

- The absolute CO_2 values are a measurement, not an estimate, and can be cited as such.
- **The comparisons between algorithms hold**, which is what answers the hypothesis of this work: all were measured the same way, on the same machine and in the same session, so any bias affects all of them equally.
- And there is a lesson that goes beyond this project: **two software footprint measurements can differ several-fold depending on how they obtained power.** A footprint report that does not state this is not verifiable.

5.3 Magnitude of the results

The emissions of a single run are tiny. The sum of all the configurations measured in this benchmark amounts to 2.060e-06 kgCO_2eq. Presenting only that number would invite the conclusion that the problem is irrelevant, and that would be the wrong reading: software runs at scale.

- The worst algorithm measured (`bubble_sort`, number of elements = 8,000) repeated 1,000,000 times, what a medium-sized web service calls over a few months, emits 0.621 kgCO_2eq, equivalent to 4 km driven by car, 179 full smartphone charges.
- The worst algorithm measured (`bubble_sort`, number of elements = 8,000) repeated 1,000,000,000 times, the scale of a massive service, emits 620.941 kgCO_2eq, equivalent to 3,991 km driven by car, 179,463 full smartphone charges.

Equivalences used:

Unit	Factor	Source / calculation
km driven by car	0.15557 $kgCO_2eq$ each	7.9 L/100 km (average light-vehicle consumption in Argentina) × 1.9693 $kgCO_2$/L (Grade 2 (Premium) gasoline with 12% bioethanol, fossil CO_2)
full smartphone charges	0.00346 $kgCO_2eq$ each	~10 Wh battery charged from the grid: 0.010 kWh × 0.346 $kgCO_2eq$/kWh (EF 2025)

6. Offsetting

One way to put the emissions in perspective is to ask how much nature would be needed to absorb them. The following annual absorption values are adopted as a reference; this work did not measure them: they are reference values stated as assumptions.

Tree	Annual absorption
Mature tree	300 $kgCO_2eq$ per year
Newly planted tree	30 $kgCO_2eq$ per year

6.1 Trees needed to offset what was measured

The full benchmark emitted 2.060e-06 $kgCO_2eq$ (section 5.3). Offsetting it requires 6.867e-09 mature trees: in practice, none. A single mature tree absorbs in one year what this benchmark emits when repeated 145,622,141 times.

The figure only becomes meaningful at real-world usage scale. With the worst algorithm measured (`bubble_sort`, number of elements = 8,000):

Usage scale	CO_2	Mature trees	Newly planted trees
1,000,000 runs	0.621 $kgCO_2eq$	2.1e-03	0.021
1,000,000,000 runs	620.941 $kgCO_2eq$	2.07	20.70

And the other way around: how many runs a single tree can offset in one year, depending on which algorithm is chosen to do the same work.

Tree	`bubble_sort`	`timsort_nativo`
Mature tree	483,137,915	1,259,707,146,246
Newly planted tree	48,313,792	125,970,714,625

Solving the same problem with `timsort_nativo` instead of `bubble_sort` makes the same tree cover 2,607 times more runs.

6.2 Processing hours offset by one tree

While the machine is computing, the emission rate is practically the same for the 11 algorithms measured: what changes between them is how long they sustain it, not its intensity. The median for this run is 1.073 grams of CO_2 per hour of processing.

Tree	Hours offset	Equivalent to	Of continuous computing
Mature tree	279,518 h	11,647 days	31.9 years
Newly planted tree	27,952 h	1,165 days	3.2 years

In other words, a newly planted tree absorbs in one year what this machine emits computing non-stop for 3.2 years. Put the other way around: a laptop working at full load 365 days a year needs 0.31 newly planted trees to become neutral.

These figures are for this machine and this power grid (sections 3.1 and 2). A data center server emits one or two orders of magnitude more per hour.

7. Conclusions

1. In Sorting algorithms (n=8000), `timsort_nativo` emits **2,607**× less CO_2 than `bubble_sort` for an identical result.
2. In Computing Fibonacci (n=32), `fib_iterativo` emits **328,443**× less CO_2 than `fib_recursivo` for an identical result.
3. In Matrix multiplication (n=300), `matmul_numpy` emits **178**× less CO_2 than `matmul_python` for an identical result.
4. In Training and inference of an ML model (KNN) (n=20000), `knn_brute` emits **13.5**× less CO_2 than `knn_ball_tree` for an identical result.

The hypothesis is confirmed. The footprint differences between equivalent algorithms are not marginal: they reach several orders of magnitude. **Choosing an appropriate algorithm is not just a performance decision, it is also an environmental decision**.

8. Reproduction

```
uv sync                          # installs the exact environment from uv.lock
uv run huella-carbono --full     # runs the benchmark and regenerates this report
```

Data inputs versioned with the project (they are not generated, they are read):

File	Contents
`datos/intensidad_carbono_argentina.json`	carbon intensity series of the Argentine grid and official cross-check factor, with their provenance
`datos/equivalencias_co2.json`	factors for translating $kgCO_2eq$ into tangible quantities (section 5.3), with their source

9. Sources and references

Power grid emissions data

1. Ember (2026), Yearly Electricity Data — with processing by Our World in Data. *Life-cycle carbon intensity of electricity generation*, series 2000–2025 for Argentina. https://ourworldindata.org/grapher/carbon-intensity-electricity (accessed on 2026-08-22). Value adopted in this work: 346.0 gCO_2eq/kWh (2025).
2. Ember. *Yearly Electricity Data* — primary source of the series above. https://ember-energy.org/data/yearly-electricity-data/.
3. Secretaría de Energía de la Nación (Argentina's national Energy Secretariat). *Grid CO2 emission factor — combined margin (w_OM = 0.75; w_BM = 0.25)*. Methodology: UNFCCC, Tool to calculate the emission factor for an electricity system, v7. http://datos.energia.gob.ar/dataset/calculo-del-factor-de-emision-de-co2-de-la-red-argentina-de-energia-electrica (file `factor_de_emision_2013_a_2023.xlsx`, accessed on 2026-08-22). Value for 2023: 343.5 gCO_2eq/kWh; cited as a cross-check, not used in the calculations.

Measurement tools

1. CodeCarbon 3.3.0 — measurement tool used in this work (section 3.4). https://github.com/mlco2/codecarbon · https://mlco2.github.io/codecarbon/. Its internal table of emission factors by country gives 354.1 gCO_2eq/kWh for Argentina, a value from 2023, which this work does not adopt for the reason explained in section 2.
2. scikit-learn 1.9.0 — provides the nearest-neighbors classifier and the synthetic dataset generator for the fourth experiment. https://scikit-learn.org/stable/.
3. Operating system power utility (macOS) — the sensor from which the processor's actual power is read on this machine (section 3.4).